

I Introduction

Un grand nombre de problèmes rencontrés en algorithmique sont des problèmes d'optimisation combinatoire. Ils consistent à rechercher, parmi un ensemble fini mais souvent gigantesque de solutions admissibles, celle qui optimise une certaine fonction objectif (minimisation ou maximisation).

Ces problèmes apparaissent dans des contextes très variés :

- recherche d'itinéraires optimaux
- planification d'activités
- allocation de ressources
- conception de réseaux

La difficulté principale réside dans l'explosion combinatoire du nombre de solutions. Une approche exhaustive devient donc rapidement impraticable même pour des instances de taille modeste.

Pour traiter ces problèmes, on distingue généralement deux familles de méthodes :

- **les méthodes exactes** (programmation dynamique, branch and bound, backtracking...) qui garantissent une solution optimale mais peuvent être coûteuses en temps de calcul ;
- **les méthodes approchées** (heuristiques, algorithmes gloutons, méta-heuristiques) qui sacrifient parfois l'optimalité au profit de l'efficacité.

II Les algorithmes de backtracking

Les algorithmes de backtracking (ou « retour sur trace ») constituent une méthode exacte de résolution de problèmes combinatoires.

L'idée générale est d'explorer l'ensemble des solutions possibles en construisant progressivement une solution candidate, et de revenir en arrière dès qu'on détecte que la solution partielle ne peut pas aboutir à une solution valide ou optimale.

Ce paradigme est particulièrement utile pour traiter des problèmes d'énumération, de recherche de solution, ou de décision sous contraintes.

II.1 Principe général

Un algorithme de backtracking suit généralement le schéma suivant :

1. Construire la solution de manière incrémentale, étape par étape.
2. À chaque étape : Vérifier si la solution partielle est encore « prometteuse » (c'est-à-dire qu'elle peut conduire à une solution valide); Si oui, continuer la construction en faisant un choix qui n'a pas déjà été exploré; Sinon, revenir en arrière et essayer une autre possibilité.
3. Lorsque la solution est complète et valide, l'enregistrer (ou la retourner si l'on cherche une seule solution).

II.2 Caractéristiques

Le backtracking repose sur deux mécanismes fondamentaux :

II.2.A Exploration systématique

Toutes les possibilités sont considérées, mais de manière structurée (souvent représentée comme un arbre de recherche). Chaque noeud de l'arbre correspond à une solution partielle, et les branches représentent les choix possibles pour l'étendre.

II.2.B Élagage (pruning)

Afin d'éviter une exploration exhaustive, on rejette rapidement les solutions partielles qui ne peuvent pas aboutir. Cela permet de réduire considérablement l'espace de recherche dans la pratique, même si la complexité reste exponentielle dans le pire cas.

Remarque : lorsque l'on ajoute une borne (par exemple en branch and bound), le backtracking peut devenir plus efficace pour les problèmes d'optimisation.

II.3 Exemple classique : Le problème des N-reines

Le problème des N-reines consiste à placer N reines sur un échiquier $N \times N$ de telle sorte qu'aucune reine ne puisse en attaquer une autre (pas deux sur la même ligne, colonne ou diagonale).

Le backtracking construit la solution en plaçant les reines une par une, ligne par ligne :

1. Placer une reine dans une case valide de la première ligne.
2. Passer à la ligne suivante et tenter de placer une reine dans une case non attaquée.
3. Si aucune case n'est possible, revenir en arrière à la ligne précédente et déplacer la reine.
4. Répéter jusqu'à ce que N reines soient placées ou que toutes les configurations aient été explorées.

Implémentation en Python :

```
from dataclasses import dataclass

@dataclass
class Pos:
    """Classe représentant une position sur le plateau"""
    x: int
    y: int

def position_reine_est_valide(pos: Pos, disposition: list[Pos], nb_reines: int) -> bool:
    """
    Vérifie si la position d'une reine est valide.

    Args:
        pos: La position à vérifier
        disposition: La disposition des autres reines sur le plateau
        nb_reines: Le nombre de reines sur le plateau

    Returns:
        True si la position est valide, False sinon
    """
    for i in range(nb_reines):
        x = disposition[i].x
        y = disposition[i].y

        if (
            y == pos.y                # même colonne
            or (x - y) == (pos.x - pos.y)  # même diagonale /
            or (x + y) == (pos.x + pos.y)  # même diagonale \
            # il n'y a jamais plusieurs reines sur la même ligne donc pas besoin de le vérifier
        ):
            return False

    return True

def poser_n_reines(N: int) -> list[Pos]:
    """
    Algorithme de backtracking pour résoudre le problème des N reines.

    Args:
        N: Le nombre de reines à placer sur le plateau NxN

    Returns:
        Une liste de positions (solution trouvée) ou None si aucune solution
    """
    if N == 0:
        return []

    solution = [None] * N # Création d'une liste de N éléments

    ligne = 0
    col = 0

    while ligne <= N:
        trouve = False

        # Essayer de placer une reine sur la ligne courante
        while col < N and not trouve:
            candidate = Pos(ligne, col)

            if position_reine_est_valide(candidate, solution, ligne):
                solution[ligne] = candidate
                trouve = True
            else:
                col += 1
```

```

if trouve:
    if ligne == N - 1:
        # Solution complète trouvée
        return solution

    # Passer à la ligne suivante
    ligne += 1
    col = 0
else:
    # Backtrack si aucune colonne n'est valide
    ligne -= 1
    if ligne >= 0:
        col = solution[ligne].y + 1

raise RuntimeError("Impossible de trouver une solution par backtracking")

```

Complexité : Dans le pire cas, l'algorithme explore toutes les configurations possibles, soit $\mathcal{O}(N!)$ (car chaque reine doit être placée sur une colonne distincte). En pratique, les tests de validité et l'élagage réduisent fortement le nombre de configurations réellement explorées.

III Les algorithmes gloutons

Les algorithmes gloutons constituent une famille de méthodes heuristiques particulièrement étudiée. Ils reposent sur une idée simple : à chaque étape, on fait un choix localement optimal dans l'espoir d'obtenir une solution globale optimale.

Cette approche est particulièrement efficace lorsque le problème satisfait une certaine structure mathématique (comme la propriété de sous-structure optimale et la propriété du choix glouton).

III.1 Principe général

Un algorithme glouton suit généralement le schéma suivant :

1. Initialiser une solution vide.
2. Tant que la solution n'est pas complète : Sélectionner l'élément qui semble le plus prometteur selon un critère local. Ajouter cet élément à la solution courante si cela reste possible.
3. Retourner la solution obtenue.

III.2 Optimalité

Pour qu'un algorithme glouton soit **correct** et donne une solution optimale, deux propriétés doivent être vérifiées :

III.2.A Sous-structure optimale

Un problème possède une sous-structure optimale lorsqu'une solution optimale au problème global peut être construite à partir de solutions optimales de sous-problèmes plus petits. Autrement dit, on peut découper le problème en sous-ensembles dont les solutions partielles contribuent directement à la solution finale. Cette propriété est également exploitée dans la programmation dynamique.

Exemple : dans le problème du plus court chemin (Dijkstra), le plus court chemin de A à C en passant par B inclut nécessairement le plus court chemin de A à B.

III.2.B Propriété du choix glouton

Un problème vérifie la propriété du choix glouton lorsqu'il est possible d'obtenir une solution optimale en effectuant, à chaque étape, un choix localement optimal (le « meilleur » selon un certain critère) sans avoir besoin de revenir sur ce choix. Cette propriété est plus restrictive que la sous-structure optimale : elle assure que l'avidité conduit directement à la solution optimale.

Exemple : dans la sélection d'activités, choisir toujours l'activité qui se termine le plus tôt ne compromet jamais la possibilité d'obtenir un ensemble maximal d'activités compatibles.

III.2.C Comment reconnaître ces propriétés ?

Il n'existe pas de recette universelle, mais plusieurs approches permettent de vérifier si un problème admet une stratégie gloutonne :

- Analyse par contre-exemple : tenter de construire une instance où le choix local échoue à produire une solution optimale. Si un tel contre-exemple existe, l'algorithme glouton n'est pas correct.
- Démonstration par induction : montrer qu'en effectuant un choix glouton à la première étape, il existe toujours une solution optimale qui contient ce choix. Cela permet souvent de justifier la validité de l'algorithme.

- Comparaison avec une solution optimale : on construit une solution optimale théorique et on montre que les décisions gloutonnes peuvent être transformées en décisions optimales sans perte de qualité.

En pratique, ces démonstrations nécessitent une bonne compréhension de la structure mathématique du problème (graphes, intervalles, contraintes combinatoires...).

III.3 Exemple classique : Problème du rendu de monnaie

Étant donné un montant à rendre et un ensemble de pièces disponibles (chaque type de pièce a une valeur et une multiplicité éventuellement infinie), le but est de rendre le montant avec un nombre minimal de pièces.

L'algorithme glouton classique consiste à :

1. Trier les valeurs des pièces par valeur décroissante : $v_1 > v_2 > \dots > v_k$.
2. Pour i de 1 à k : prendre le plus grand nombre possible de pièces de valeur v_i sans dépasser le montant restant.
3. Continuer jusqu'à ce que le montant soit rendu ou qu'il ne reste plus de pièces.

Implémentation en Python :

```
def rendre_monnaie(montant: int, valeurs: list[int]) -> list[int]:
    """
    Algorithme glouton pour rendre la monnaie.

    Args:
        montant: Le montant à rendre
        valeurs: Les valeurs des pièces (triées par ordre décroissant)

    Returns:
        Liste d'entiers représentant les pièces utilisées,
        ou None si impossible
    """
    reste = montant
    solution = []

    for v in valeurs:
        q = reste // v # Nombre de pièces de valeur v

        for _ in range(q):
            solution.append(v)

        reste -= q * v

    if reste != 0:
        raise RuntimeError("Impossible de trouver une solution par stratégie gloutonne")

    return solution
```

Complexité : Si k est le nombre de types de pièces, l'algorithme est en $O(k)$ si les valeurs sont déjà ordonnées.